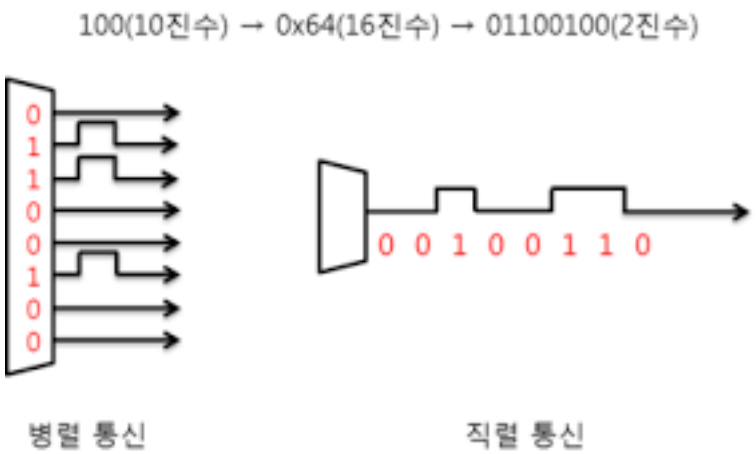
Digital IO + 시리얼통신

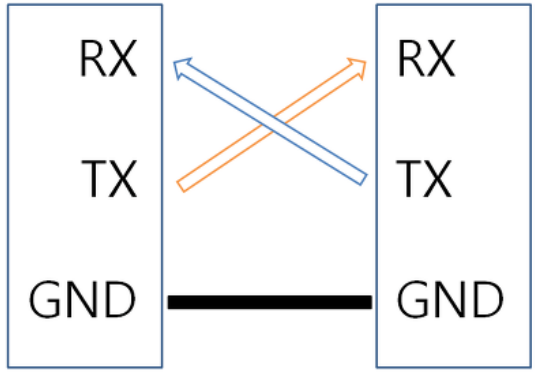
마이크로프로세서 종합 설계. 6주차.

오늘의 목표

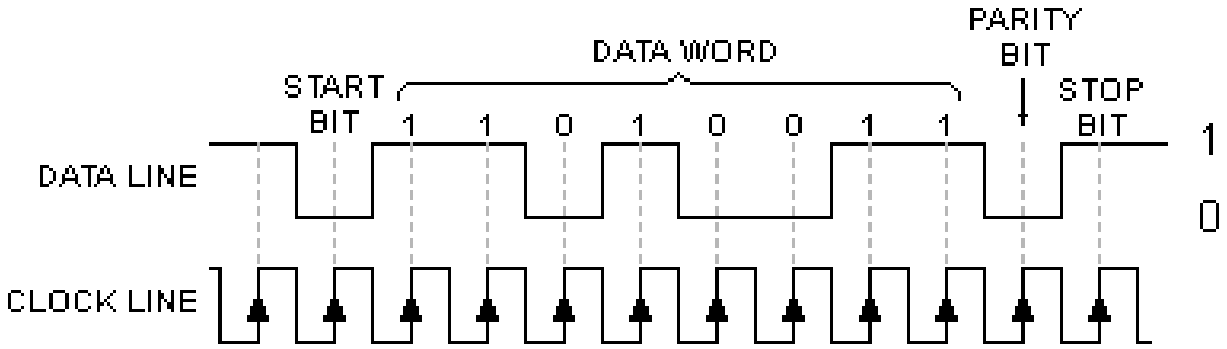
1. **시리얼통신의 이해 및 PC와의 정보 전달 실험**
2. **변수**
3. **디지털 입출력 실험**
    1. **Button +LED**
4. **외부인터럽트의 이해 및 인터럽트 실험**

마이크로프로세서와 C언어 - 시리얼통신





직렬통신의 연결



동기/비동기 통신

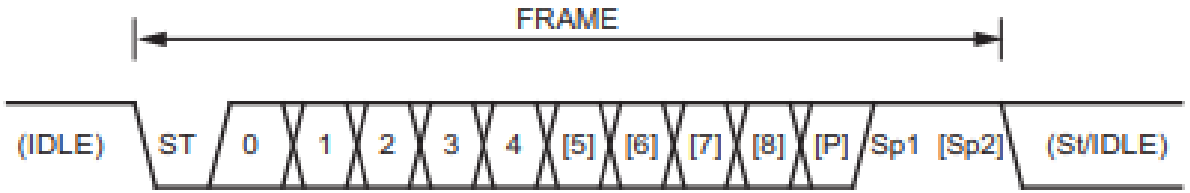
마이크로프로세서와 C언어 - 시리얼통신

- **비동기식 시리얼 통신**
  - UART 통신을 이용하기 위해서는 크게 다음의 두 가지 항목을 사전에 정의해줘야 한다.
    - **통신속도 : Baud rate**
      - Baud rate 의 단위는 bps(bits per second) : 1초당 전송하는 bit 수
      - 표준 bps: 1200, 2400, 4800, **9600**, 19200, 38400, 57600, 115200
    - **프레임사이즈 : Size of each frame field**
      - 일반적으로
        - Data bit는 1
        - Bytes site = 8 bits 사이즈로 설정
        - Stop bit는 1 bit
        - Parity bit는 0 bit로 설정
      - 통신을 사용하는 환경에 따라 미리 약속하여 사용

마이크로프로세서와 C언어 - 시리얼통신

- 시리얼통신 데이터 포멧

Figure 19-4. Frame Formats



St	Start bit, always low.
(n)	Data bits (0 to 8).
P	Parity bit. Can be odd or even.
Sp	Stop bit, always high.
IDLE	No transfers on the communication line (RxDn or TxDn). An IDLE line must be high.

참고 : 데이타시트 https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf 147p

아두이노를 이용한 시리얼통신 실험

아두이노 우노로부터 시리얼통신으로 "Hello World"라는 데이터를 확인하는 코드를 작성해줘

아두이노 우노에서 "Hello World" 데이터를 시리얼 통신으로 전송하고, C 언어로 이를 수신하는 코드

---

1. 아두이노 코드 (Arduino IDE에서 업로드)

c 복사 편집

```c
void setup() {
    Serial.begin(9600); // 시리얼 통신 시작 (9600bps)
}

void loop() {
    Serial.println("Hello World"); // "Hello World" 메시지 전송
    delay(1000); // 1초 대기 후 반복
}
```

설명

- `Serial.begin(9600);` → 9600bps 속도로 시리얼 통신을 시작합니다.
- `Serial.println("Hello World");` → `"Hello World"` 문자열을 전송합니다.
- `delay(1000);` → 1초 간격으로 데이터를 송신합니다.

```c
void setup()
{
    Serial.begin(9600); // 시리얼 통신 시작 (9600bps)
}

void loop()
{
    // "Hello World" 메시지 전송
    Serial.println("Hello World");
    delay(1000); // 1초 대기 후 반복
}
```

마이크로프로세서와 C언어 - ASCII

Dec	Hx	Oct	Char	
0	0	000	NUL	(null)
1	1	001	SOH	(start of heading)
2	2	002	STX	(start of text)
3	3	003	ETX	(end of text)
4	4	004	EOT	(end of transmission)
5	5	005	ENQ	(enquiry)
6	6	006	ACK	(acknowledge)
7	7	007	BEL	(bell)
8	8	010	BS	(backspace)
9	9	011	TAB	(horizontal tab)
10	A	012	LF	(NL line feed, new line)
11	B	013	VT	(vertical tab)
12	C	014	FF	(NP form feed, new page)
13	D	015	CR	(carriage return)
14	E	016	SO	(shift out)
15	F	017	SI	(shift in)
16	10	020	DLE	(data link escape)
17	11	021	DC1	(device control 1)
18	12	022	DC2	(device control 2)
19	13	023	DC3	(device control 3)
20	14	024	DC4	(device control 4)
21	15	025	NAK	(negative acknowledge)
22	16	026	SYN	(synchronous idle)
23	17	027	ETB	(end of trans. block)
24	18	030	CAN	(cancel)
25	19	031	EM	(end of medium)
26	1A	032	SUB	(substitute)
27	1B	033	ESC	(escape)
28	1C	034	FS	(file separator)
29	1D	035	GS	(group separator)
30	1E	036	RS	(record separator)
31	1F	037	US	(unit separator)

Dec	Hx	Oct	Char
32	20	040	Space
33	21	041	!
34	22	042	"
35	23	043	#
36	24	044	$
37	25	045	%
38	26	046	&
39	27	047	'
40	28	050	(
41	29	051	)
42	2A	052	*
43	2B	053	+
44	2C	054	,
45	2D	055	-
46	2E	056	.
47	2F	057	/
48	30	060	0
49	31	061	1
50	32	062	2
51	33	063	3
52	34	064	4
53	35	065	5
54	36	066	6
55	37	067	7
56	38	070	8
57	39	071	9
58	3A	072	:
59	3B	073	;
60	3C	074	<
61	3D	075	=
62	3E	076	>
63	3F	077	?

Dec	Hx	Oct	Char
64	40	100	@
65	41	101	A
66	42	102	B
67	43	103	C
68	44	104	D
69	45	105	E
70	46	106	F
71	47	107	G
72	48	110	H
73	49	111	I
74	4A	112	J
75	4B	113	K
76	4C	114	L
77	4D	115	M
78	4E	116	N
79	4F	117	O
80	50	120	P
81	51	121	Q
82	52	122	R
83	53	123	S
84	54	124	T
85	55	125	U
86	56	126	V
87	57	127	W
88	58	130	X
89	59	131	Y
90	5A	132	Z
91	5B	133	[
92	5C	134	\
93	5D	135	]
94	5E	136	^
95	5F	137	_

Dec	Hx	Oct	Char
96	60	140	`
97	61	141	a
98	62	142	b
99	63	143	c
100	64	144	d
101	65	145	e
102	66	146	f
103	67	147	g
104	68	150	h
105	69	151	i
106	6A	152	j
107	6B	153	k
108	6C	154	l
109	6D	155	m
110	6E	156	n
111	6F	157	o
112	70	160	p
113	71	161	q
114	72	162	r
115	73	163	s
116	74	164	t
117	75	165	u
118	76	166	v
119	77	167	w
120	78	170	x
121	79	171	y
122	7A	172	z
123	7B	173	{
124	7C	174	|
125	7D	175	}
126	7E	176	~
127	7F	177	DEL

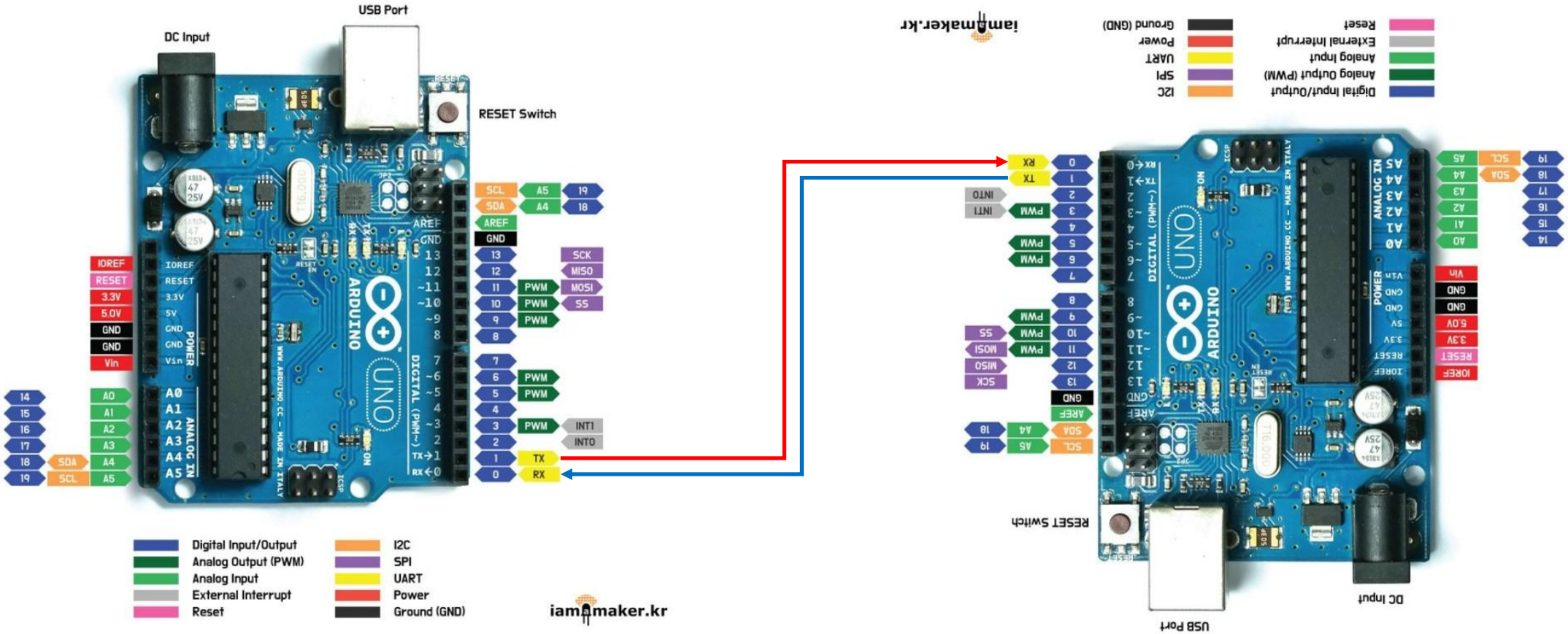
128 Ç	161 í	193 ┴	225 ß
129 ü	162 ó	194 ┬	226 Γ
130 é	163 ú	195 ├	227 π
131 â	164 ñ	196 ─	228 Σ
132 ä	165 Ñ	197 ┼	229 σ
133 à	166 ª	198 ╞	230 µ
134 å	167 º	199 ╟	231 τ
135 ç	168 ¿	200 ╚	232 Φ
136 ê	169 ⌐	201 ╔	233 Θ
137 ë	170 ¬	202 ╩	234 Ω
138 è	171 ½	203 ╦	235 δ
139 ï	172 ¼	204 ╠	236 ∞
140 î	173 ¡	205 ═	237 φ
141 ì	174 «	206 ╬	238 ε
142 Ä	175 »	207 ╧	239 ∩
143 Å	176 ░	208 ╨	240 ≡
144 É	177 ▒	209 ╤	241 ±
145 æ	178 ▓	210 ╥	242 ≥
146 Æ	179 │	211 ╙	243 ≤
147 ô	180 ┤	212 ╘	244 ⌠
148 ö	181 ╡	213 ╒	245 ⌡
149 ò	182 ╢	214 ╓	246 ÷
150 û	183 ╖	215 ╫	247 ≈
151 ù	184 ╕	216 ╪	248 °
152 _	185 ╣	217 ┘	249 ·
153 Ö	186 ║	218 ┌	250 ·
154 Ü	187 ╗	219 █	251 √
156 £	188 ╝	220 ▄	252 ⁿ
157 ¥	189 ╜	221 ▌	253 ²
158 _	190 ╛	222 ▐	254 ■
159 ƒ	191 ┐	223 ▀	255
160 á	192 └	224 α	

마이크로프로세서와 C언어 - ASCII

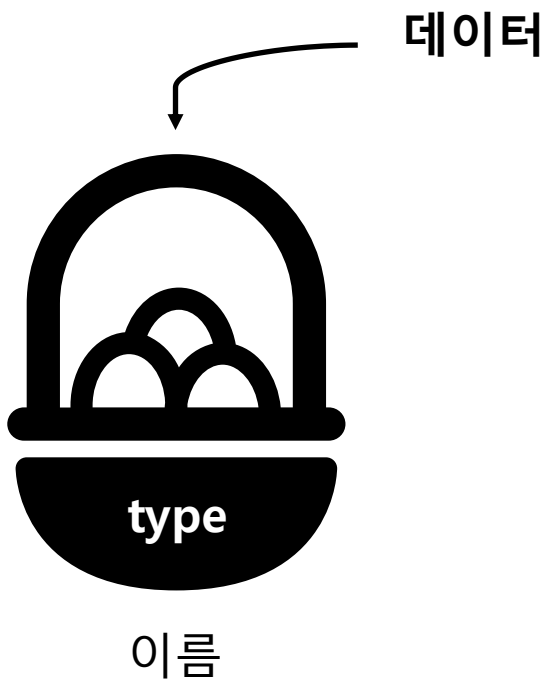
Dec	Hx	Oct	Char	Dec	Hx	Oct	Char
65	41	101	A	97	61	141	a
66	42	102	B	98	62	142	b
67	43	103	C	99	63	143	c
68	44	104	D	100	64	144	d
69	45	105	E	101	65	145	e
70	46	106	F	102	66	146	f
71	47	107	G	103	67	147	g
72	48	110	H	104	68	150	h
73	49	111	I	105	69	151	i
74	4A	112	J	106	6A	152	j
75	4B	113	K	107	6B	153	k
76	4C	114	L	108	6C	154	l
77	4D	115	M	109	6D	155	m
78	4E	116	N	110	6E	156	n
79	4F	117	O	111	6F	157	o
80	50	120	P	112	70	160	p
81	51	121	Q	113	71	161	q
82	52	122	R	114	72	162	r
83	53	123	S	115	73	163	s
84	54	124	T	116	74	164	t
85	55	125	U	117	75	165	u
86	56	126	V	118	76	166	v
87	57	127	W	119	77	167	w
88	58	130	X	120	78	170	x
89	59	131	Y	121	79	171	y
90	5A	132	Z	122	7A	172	z

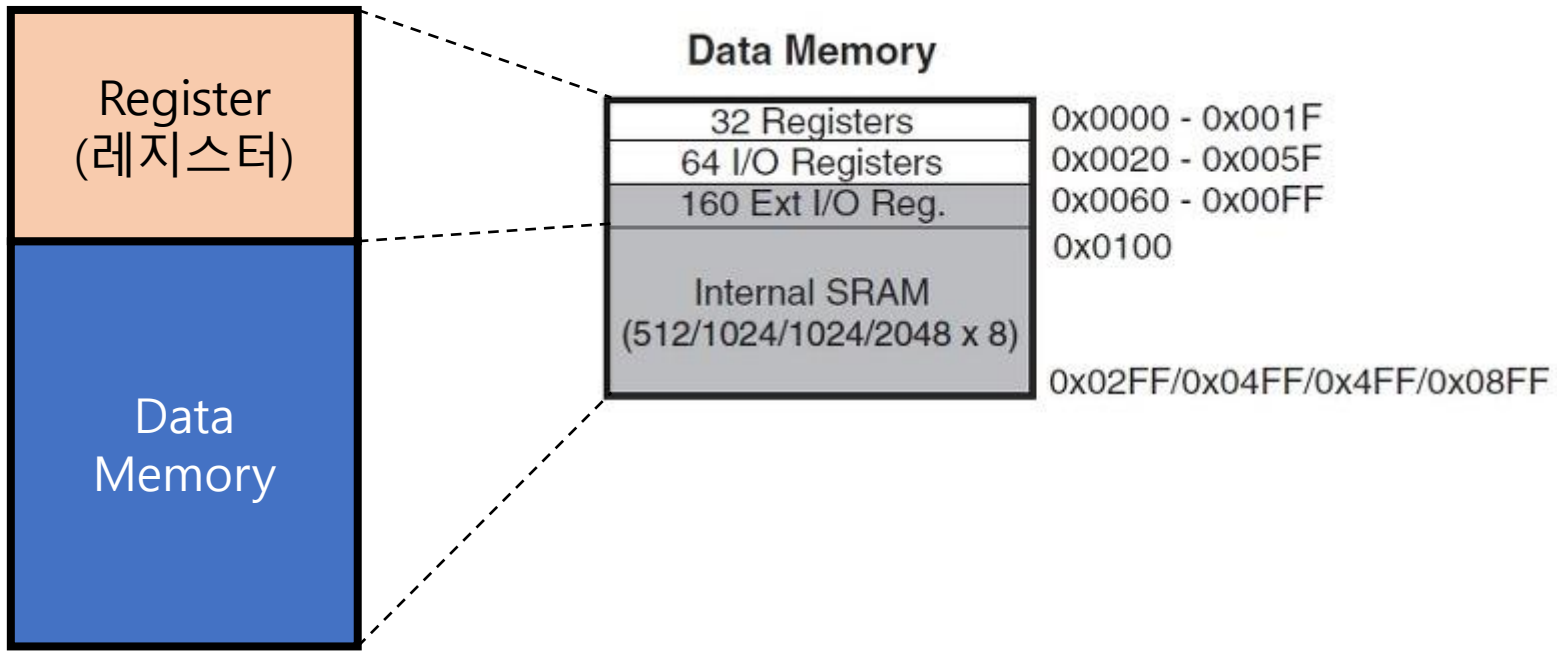
0x48
0x65
0x6C
0x6C
0x6F

아두이노를 이용한 시리얼통신 실험

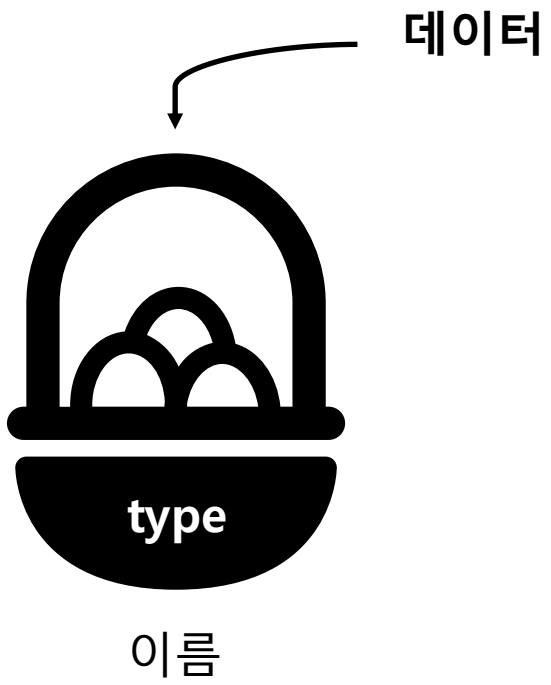


마이크로프로세서와 C언어 - 변수



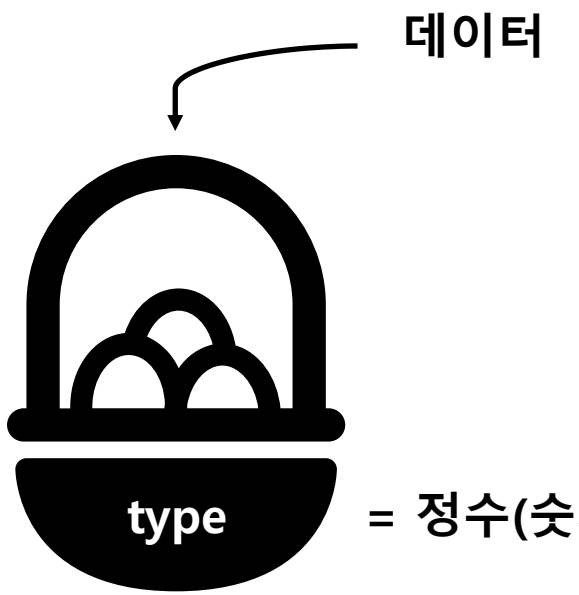


마이크로프로세서와 C언어 – 변수의 종류



type	설명	사이즈(Byte)
char	단일 문자를 저장하거나 -128에서 127 사이의 작은 정수를 저장하는 데 사용	1
unsinged char	0에서 255 사이의 값을 저장할 수 있는 8비트 부호 없는 정수	1
byte	0에서 255 사이의 값을 저장할 수 있는 8비트 부호 없는 정수	
int	정수를 저장하는 데 사용, 아두이노에서는 보통 -32,768에서 32,767 사이의 값을 저장.	2
unsigned int	부호 없는 정수를 저장, 0에서 65,535 사이의 값을 저장	2
long	더 큰 정수를 저장할 수 있으며, -2,147,483,648에서 2,147,483,647 사이의 값을 저장	4
unsigned long	부호 없는 더 큰 정수를 저장하며, 0에서 4,294,967,295 사이의 값을 저장	4
short	-32,768에서 32,767 사이의 값을 저장할 수 있는 작은 정수	2
float	부동소수점 숫자를 저장하는 데 사용, 대략 -3.4E+38에서 +3.4E+38 사이의 값을 6~7자리의 정밀도로 저장	4
double	아두이노에서는 float와 동일하게 처리되는 경우가 많으며, 일부 아두이노 보드(예: SAMD 기반의 보드)에서는 더 큰 범위와 정밀도를 제공	4
bool	true 또는 false 값을 저장하는 데 사용	1

마이크로프로세서와 C언어 - 변수



= 정수(숫자)

이름 = number

- int number = 10 ;
- unsigned int number = 10 ;
- unsigned int number = -10 ;
- long number = -10 ;
- unsigned long number = -10 ;

정수형 변수에 데이터를 저장하고 시리얼 통신으로 데이터 확인

You

number라고 이름 붙인 정수형 변수에 데이터 10을 넣고 이를 시리얼통신으로 확인하는 아두이노 코드를 작성하세요.

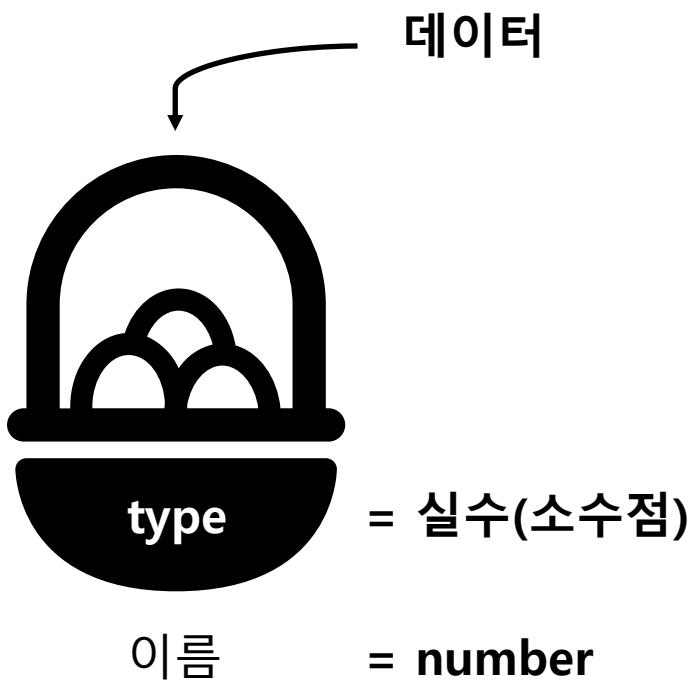
ChatGPT

아두이노에서 특정한 데이터를 시리얼 통신을 통해 확인하기 위해서는 시리얼 통신을 초기화하고, 해당 데이터를 시리얼 모니터로 전송하는 코드를 작성해야 합니다. 아래의 코드 예제는 정수형 변수 `number`에 `10`을 할당하고, 이를 시리얼 통신을 통해 전송하여 시리얼 모니터에서 확인할 수 있도록 하는 아두이노 스케치입니다.

cpp Copy code

```cpp
void setup() {
  // 시리얼 통신을 시작합니다. 보드레이트는 9600bps로 설정합니다.
  Serial.begin(9600);
}

void loop() {
  // 정수형 변수 number를 선언하고 10을 할당합니다.
  int number = 10;

  // number 변수의 값을 시리얼 모니터로 전송합니다.
  Serial.println(number);

  // 너무 빠르게 데이터가 전송되지 않도록 1초 간격으로 딜레이를 줍니다.
  delay(1000);
}
```

마이크로프로세서와 C언어 - 변수



- **float number = 0.13 ;**
- **double number = -10.123 ;**

실수형 변수에 데이터를 저장하고 시리얼 통신으로 데이터 확인

You

number라고 이름 붙인 실수형 변수에 데이터 10.12을 넣고 이를 시리얼통신으로 확인하는 아두이노 코드를 작성하세요.

ChatGPT

실수형 변수에 데이터를 할당하고, 이를 시리얼 통신을 통해 확인할 수 있는 아두이노 코드는 아래와 같습니다. 이 예제에서는 `number`라는 이름의 실수형(float) 변수에 `10.12`를 할당하고, 이 값을 시리얼 모니터를 통해 출력합니다.

cpp Copy code

```cpp
void setup() {
  // 시리얼 통신을 시작합니다. 보드레이트는 9600bps로 설정합니다.
  Serial.begin(9600);
}

void loop() {
  // 실수형 변수 number를 선언하고 10.12를 할당합니다.
  float number = 10.12;

  // number 변수의 값을 시리얼 모니터로 전송합니다.
  Serial.println(number);

  // 너무 빠르게 데이터가 전송되지 않도록 1초 간격으로 딜레이를 줍니다.
  delay(1000);
}
```

실험 – 변수를 담는 메모리 사이즈를 알아봅시다.

sizeof

You
변수형에 따른 메모리 사이즈를 시리얼통신으로 확인하는 코드를 만들어 줘

ChatGPT
아두이노에서 다양한 데이터 타입의 메모리 사이즈를 확인하고 이를 시리얼 통신을 통해 출력하는 코드를 작성할 수 있습니다. 이 코드는 `sizeof` 연산자를 사용하여 각 데이터 타입의 메모리 크기를 바이트 단위로 계산하고, 그 결과를 시리얼 모니터에 출력합니다.

cpp Copy code

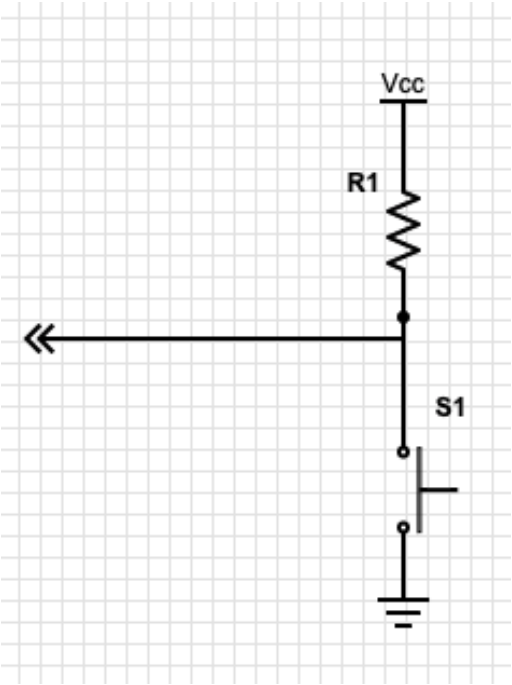
```cpp
void setup() {
  // 시리얼 통신을 시작합니다. 보드레이트는 9600bps로 설정합니다.
  Serial.begin(9600);

  // 데이터 타입별 메모리 사이즈를 시리얼 모니터로 출력합니다.
  Serial.print("char 크기: ");
  Serial.println(sizeof(char)); // char의 메모리 크기 출력

  Serial.print("int 크기: ");
  Serial.println(sizeof(int)); // int의 메모리 크기 출력

  Serial.print("unsigned int 크기: ");
  Serial.println(sizeof(unsigned int)); // unsigned int의 메모리 크기 출력

  Serial.print("long 크기: ");
  Serial.println(sizeof(long)); // long의 메모리 크기 출력

  Serial.print("unsigned long 크기: ");
  Serial.println(sizeof(unsigned long)); // unsigned long의 메모리 크기 출력

  Serial.print("float 크기: ");
  Serial.println(sizeof(float)); // float의 메모리 크기 출력

  Serial.print("double 크기: ");
  Serial.println(sizeof(double)); // double의 메모리 크기 출력 (아두이노에서 double은 flo

  Serial.print("bool 크기: ");
  Serial.println(sizeof(bool)); // bool의 메모리 크기 출력
}

void loop() {
  // 이 예제에서 loop() 함수는 사용하지 않습니다.
}
```

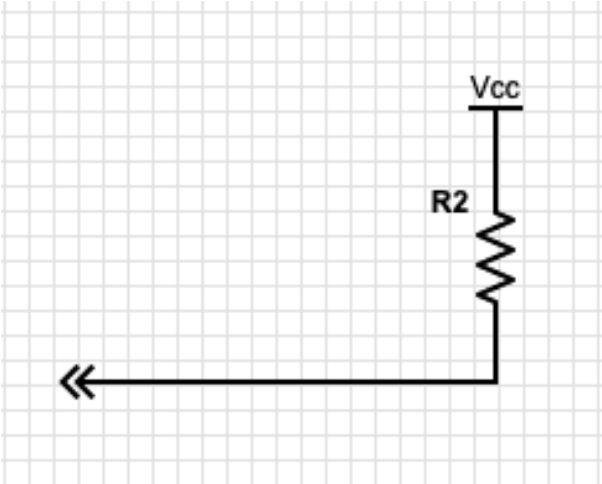
실험 – 1부터 10까지 곱한 결과는?

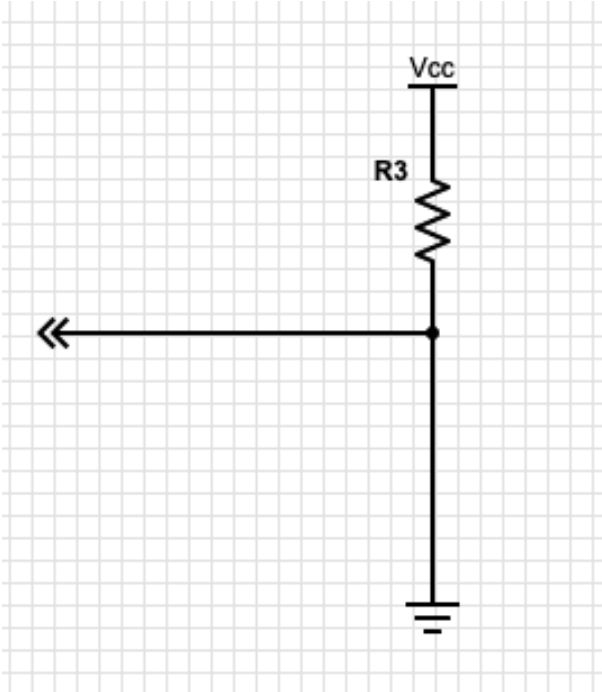
cpp Copy code

```cpp
void setup() {
  // 시리얼 통신을 시작합니다. 보드레이트는 9600bps로 설정합니다.
  Serial.begin(9600);

  // 1부터 10까지 곱한 결과를 직접 계산하고 변수에 할당합니다.
  unsigned long result = 1 * 2 * 3 * 4 * 5 * 6 * 7 * 8 * 9 * 10;

  // 계산된 결과를 시리얼 모니터로 출력합니다.
  Serial.print("1부터 10까지 곱한 결과: ");
  Serial.println(result);
}

void loop() {
  // 이 예제에서 loop() 함수는 사용하지 않습니다.
}
```

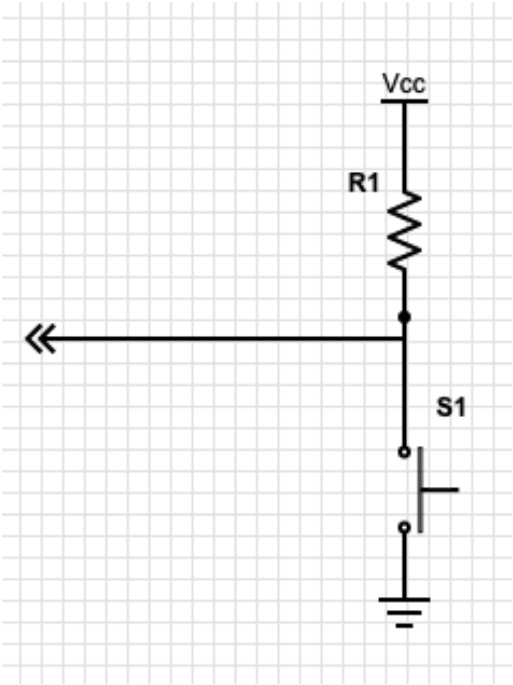
버튼 입력 실험(Digital Input)

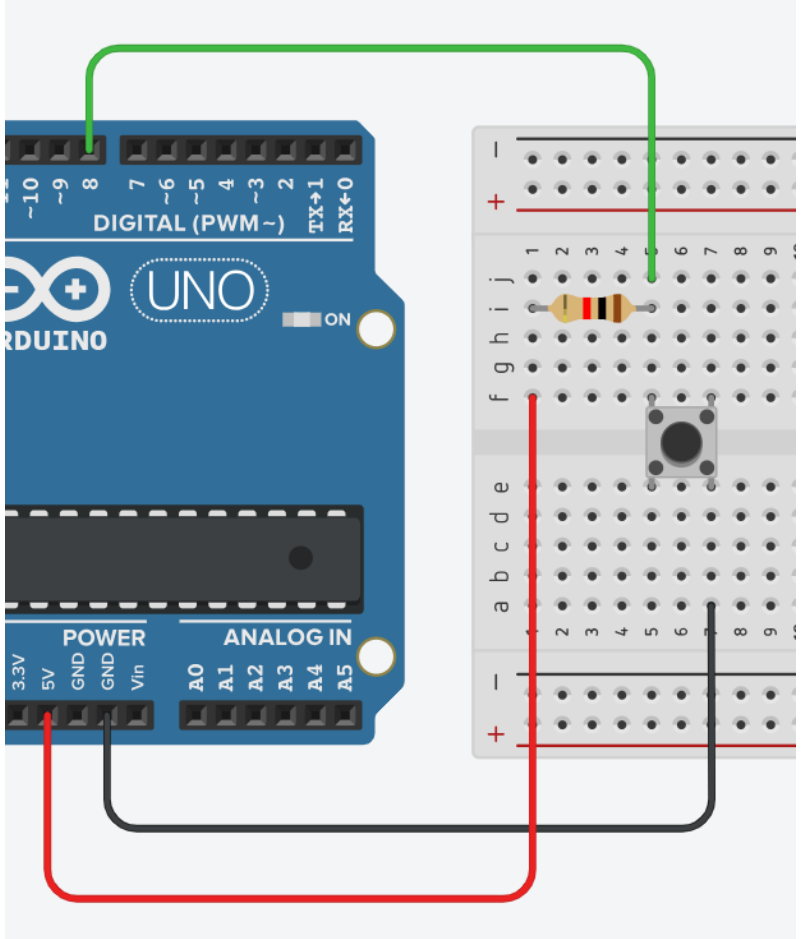




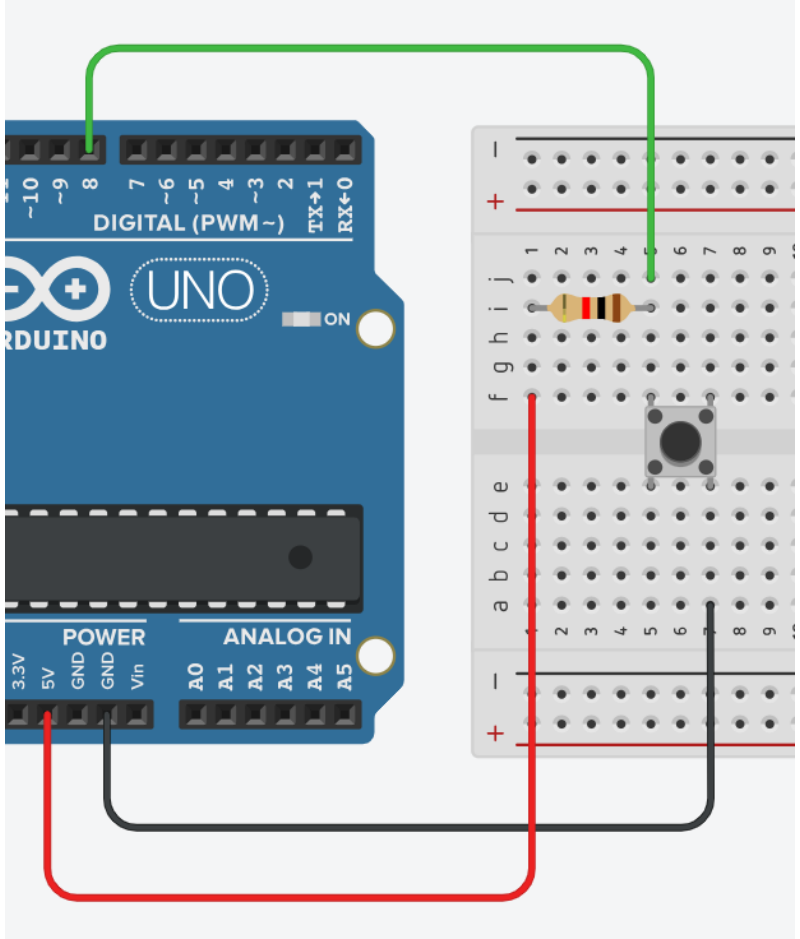


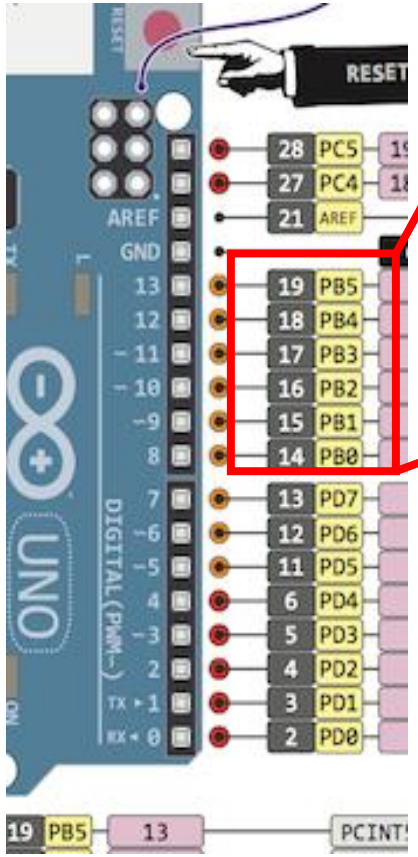
버튼 입력 실험(Digital Input)





버튼 입력 실험(Digital Input)





13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

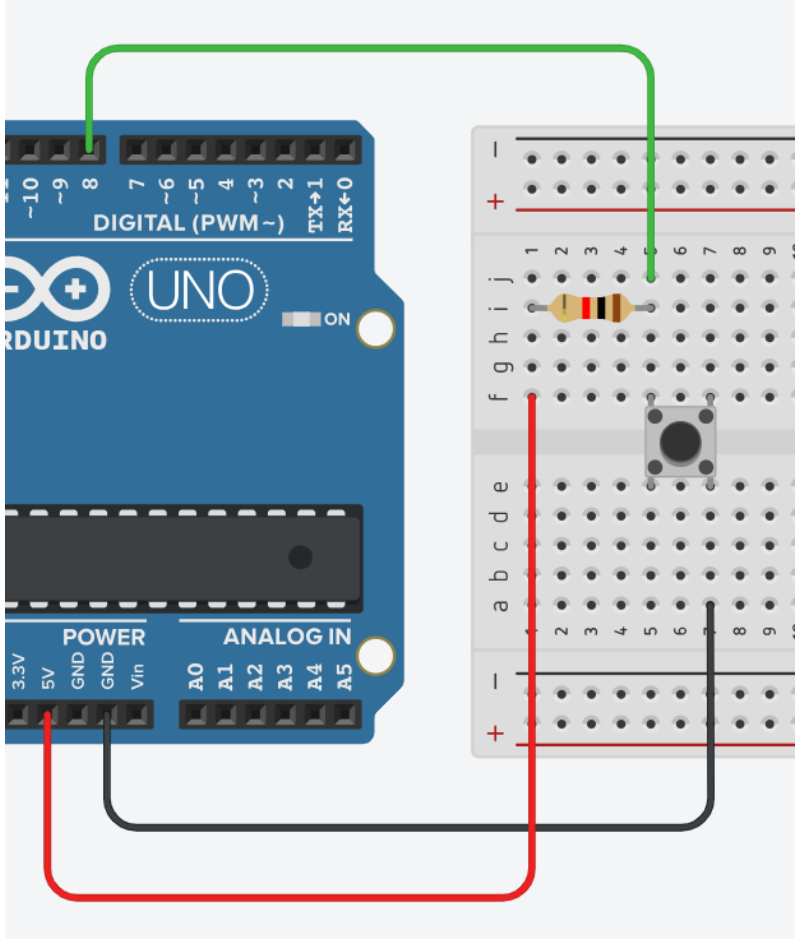
13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.4 PINB – The Port B Input Pins Address

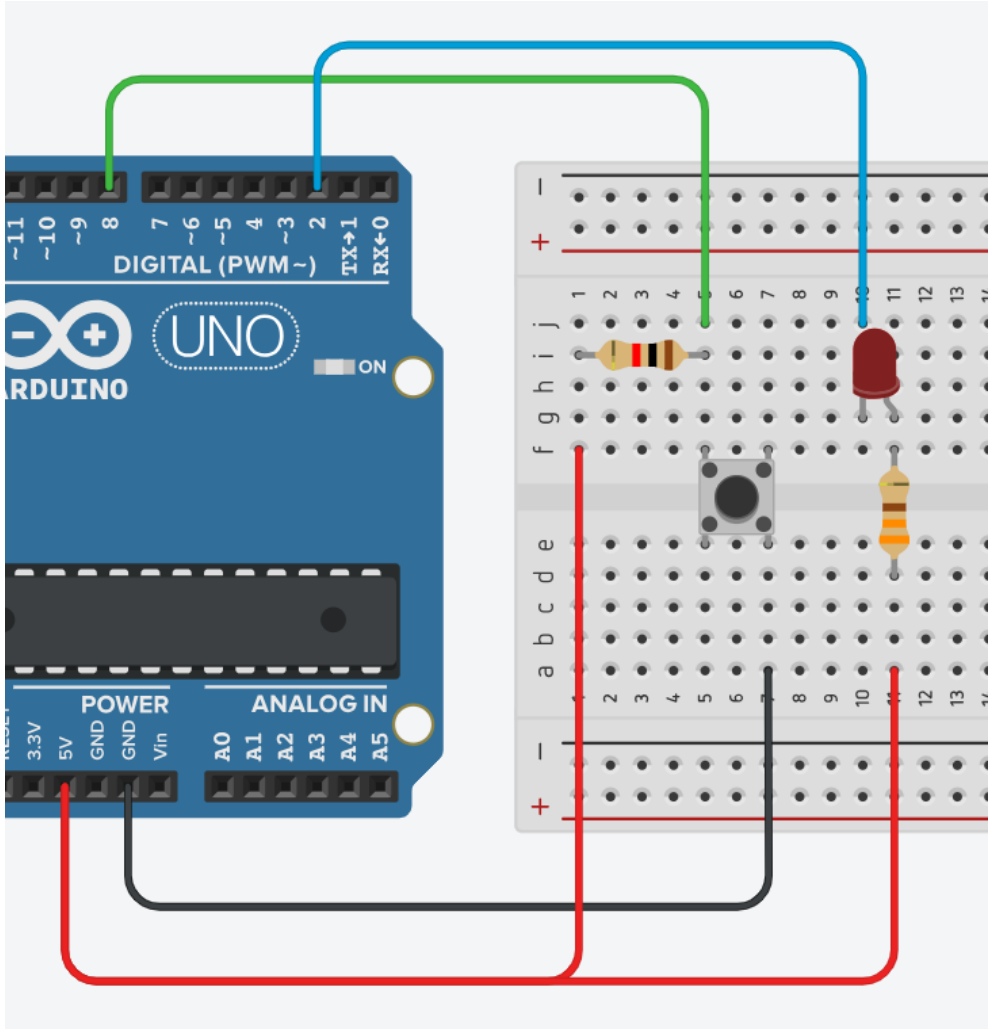
Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

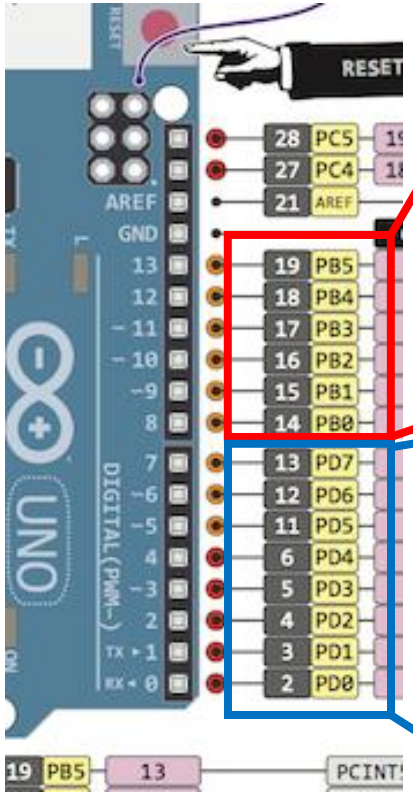
버튼 입력 실험(Digital Input)




```
void setup()
{
  DDRB = B00000000 ;
  Serial.begin(9600) ;
}

void loop()
{
  Serial.println(PINB) ;
}
```

버튼 입력 실험(Digital Input)





13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.4 PINB – The Port B Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

13.4.8 PORTD – The Port D Data Register

Bit	7	6	5	4	3	2	1	0	
0x0B (0x2B)	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0	PORTD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

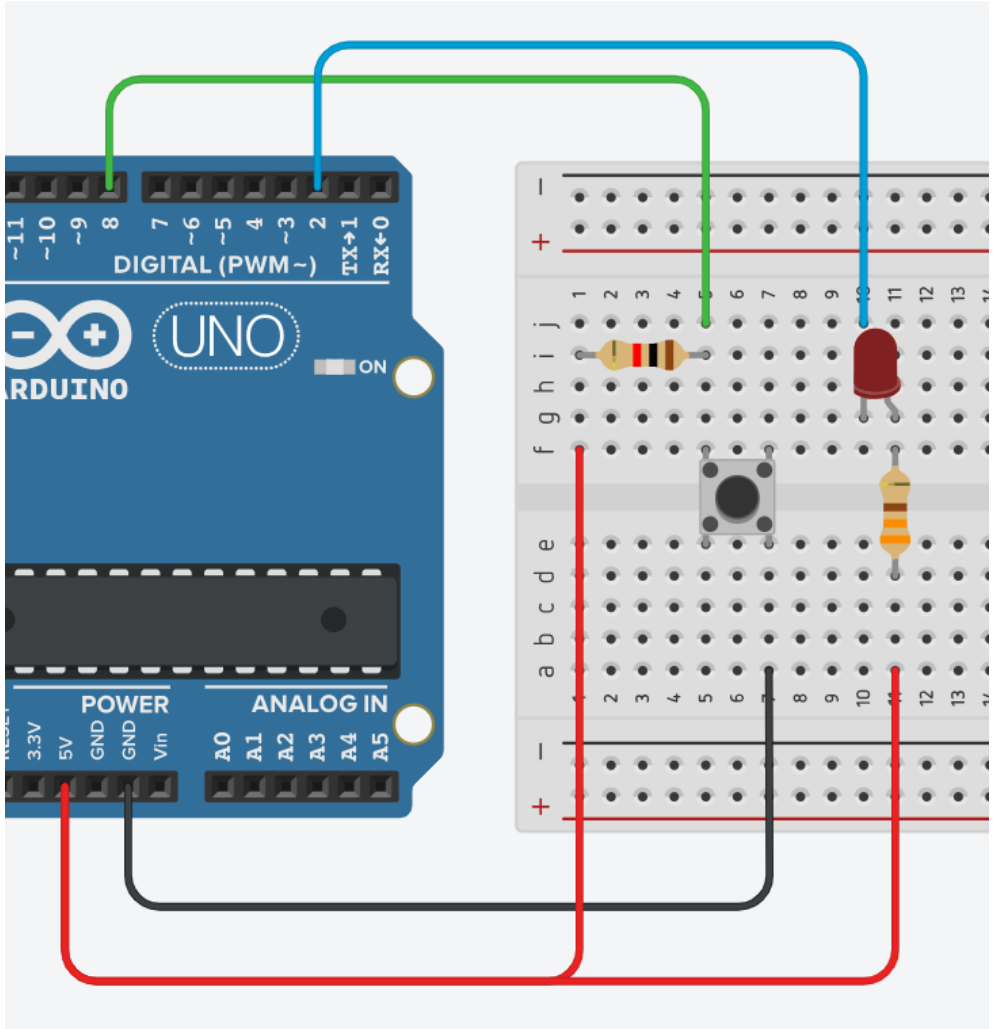
13.4.9 DDRD – The Port D Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x0A (0x2A)	DDD7	DDD6	DDD5	DDD4	DDD3	DDD2	DDD1	DDD0	DDRD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.10 PIND – The Port D Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x09 (0x29)	PIND7	PIND6	PIND5	PIND4	PIND3	PIND2	PIND1	PIND0	PIND
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

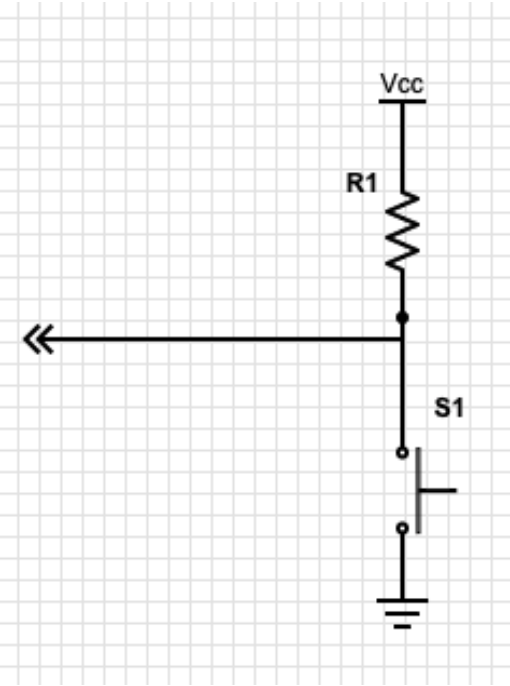
버튼 입력 실험(Digital Input)

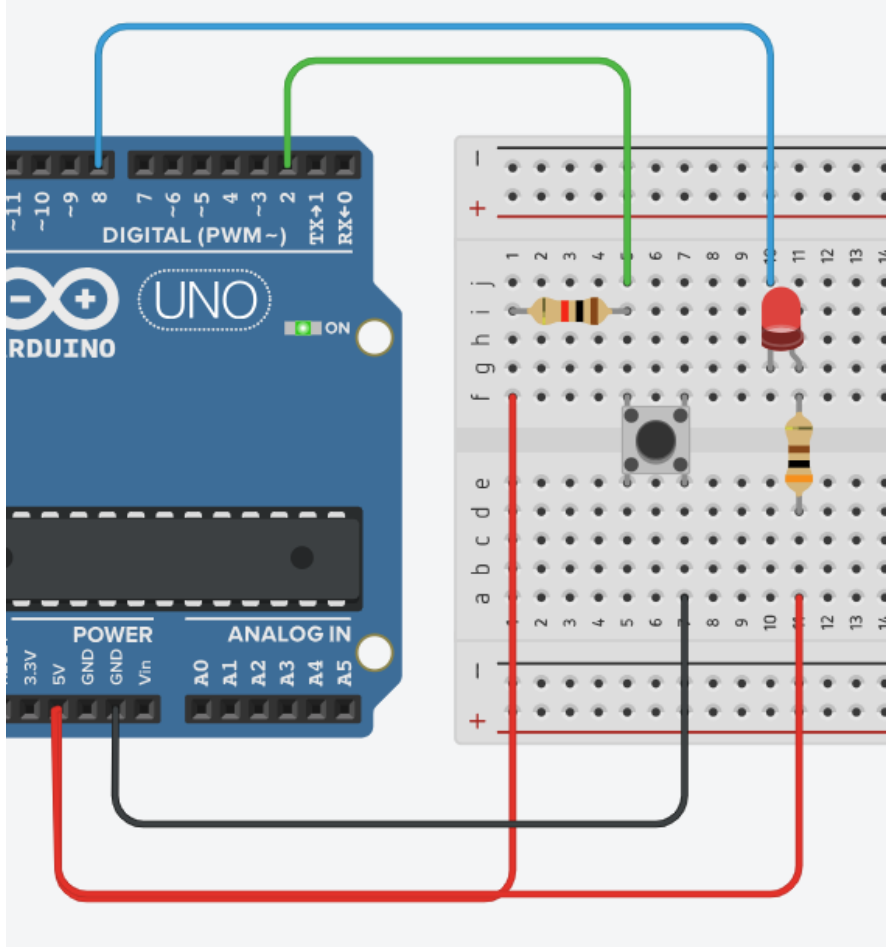



```
void setup()
{
  DDRB = B00000000 ;
  DDRD = B00000100 ;

  Serial.begin(9600) ;
}

void loop()
{
  Serial.println(PINB) ;

  if( PINB == B00000001 )
  {
    //버튼이 눌리지 않음
    PORTD = B0000100 ;          //LED 꺼짐
  }
  else
  {
    //버튼이 눌림
    PORTD = B0000000 ;          //LED 켜짐
  }
}
```

외부 인터럽트(External Interrupt)

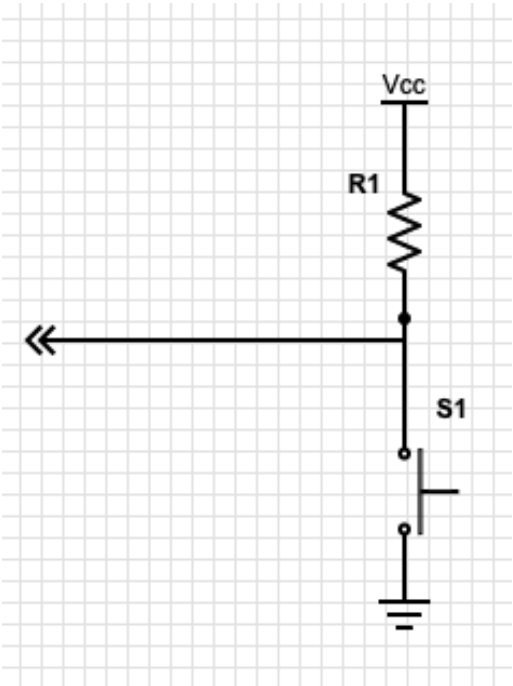
- **폴링** vs 인터럽트

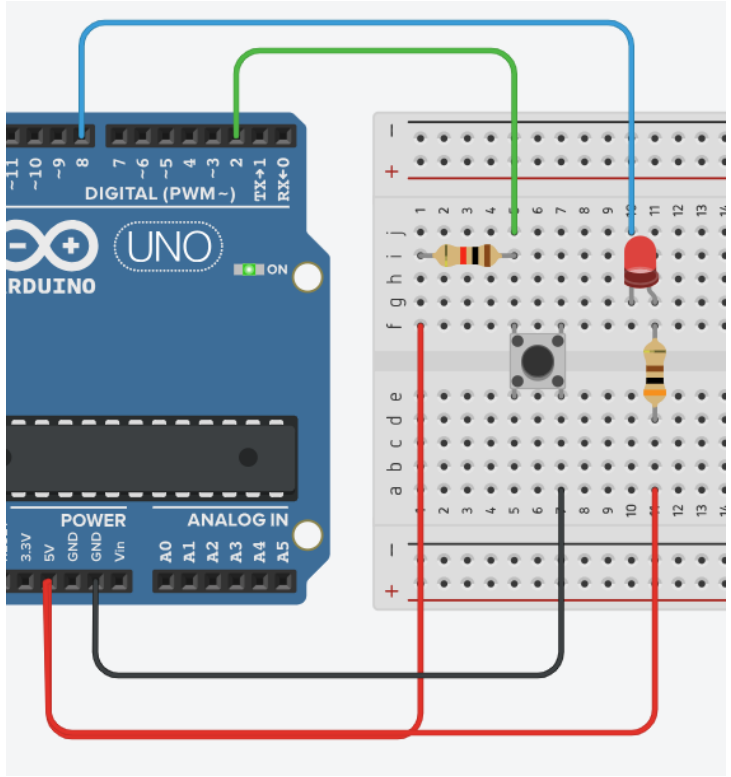




외부 인터럽트(External Interrupt)

- **폴링** vs 인터럽트

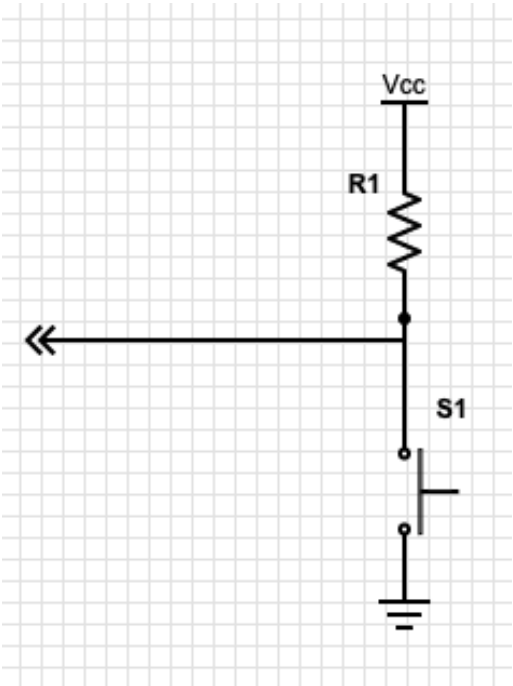


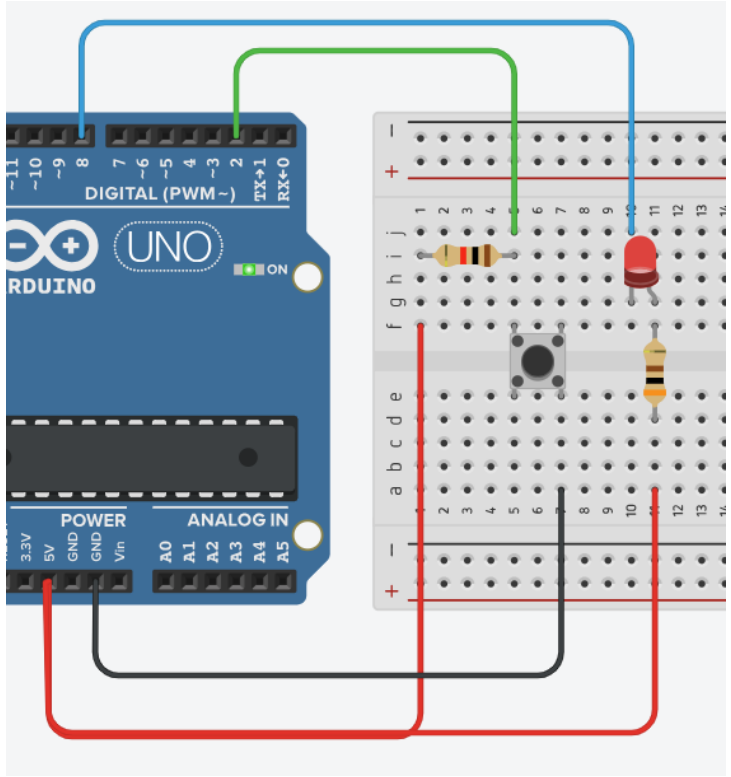



```
void setup()
{
  pinMode(2, INPUT) ;
  pinMode(8, OUTPUT) ;
}

void loop()
{
  int input = digitalRead(2) ;

  if( input == 0  )
  {
      digitalWrite(8, 0) ;
  }
  else
  {
       digitalWrite(8, 1) ;
  }
}
```

외부 인터럽트(External Interrupt)

- **폴링** vs 인터럽트

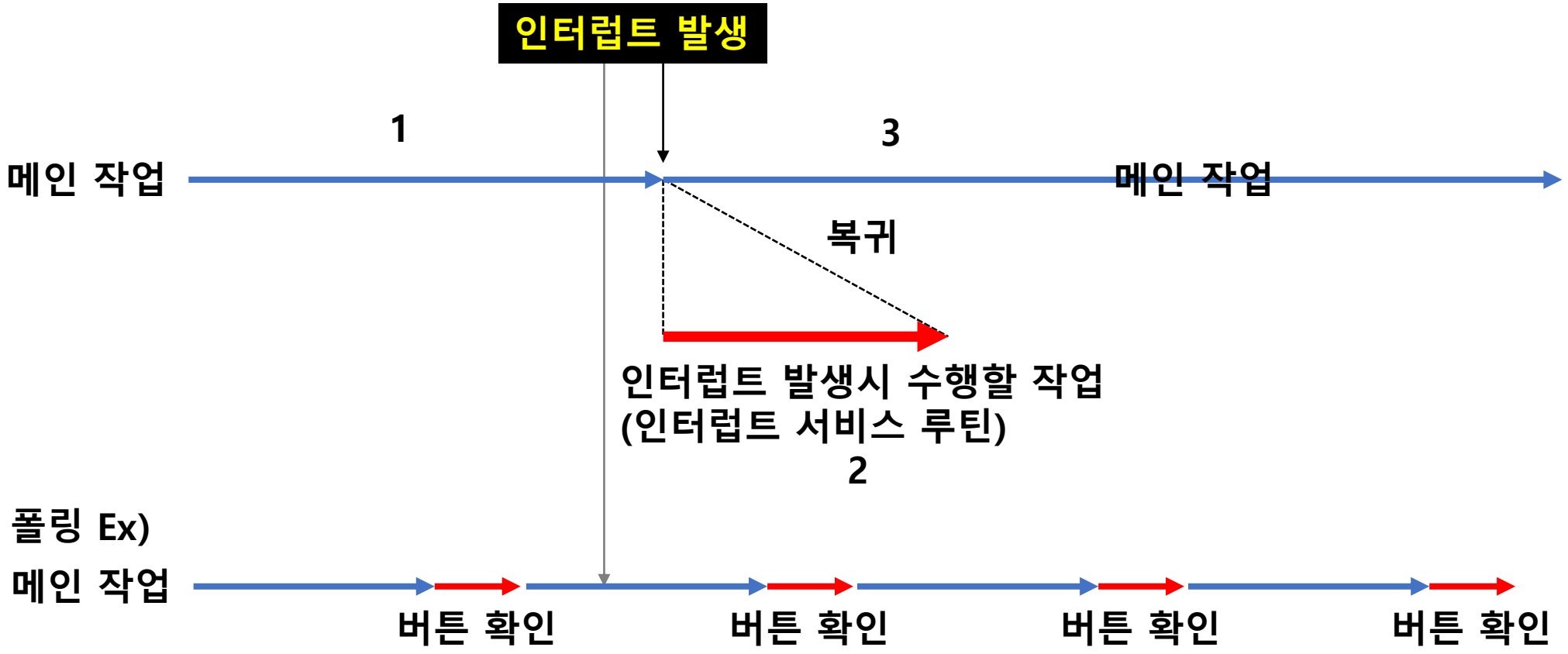





```
void setup()
{
  pinMode(2, INPUT) ;
  pinMode(8, OUTPUT) ;

  Serial.begin(9600) ;
}

void loop()
{

  digitalWrite(8, 0) ;
  delay(1000) ;

  digitalWrite(8, 1) ;
  delay(1000) ;

  int input = digitalRead(2) ;
  if( input == 0 )
  {
      Serial.println("key") ;
  }
}
```

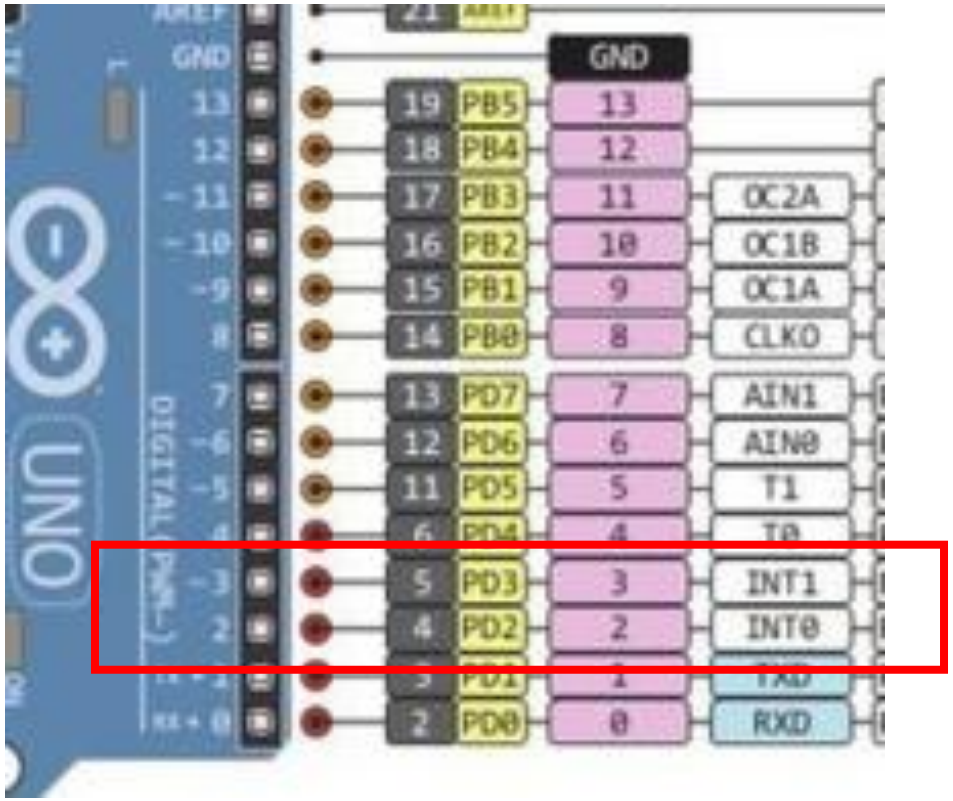
외부 인터럽트(External Interrupt)

- 폴링 vs **인터럽트**



외부 인터럽트(External Interrupt)

- 폴링 vs **인터럽트**

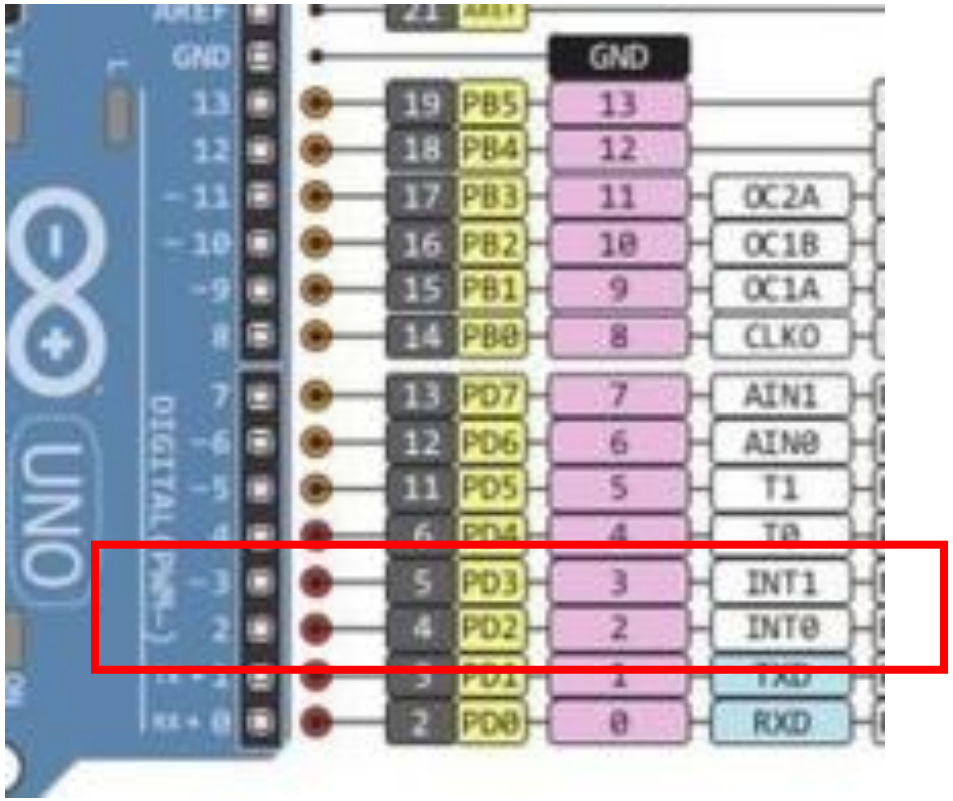


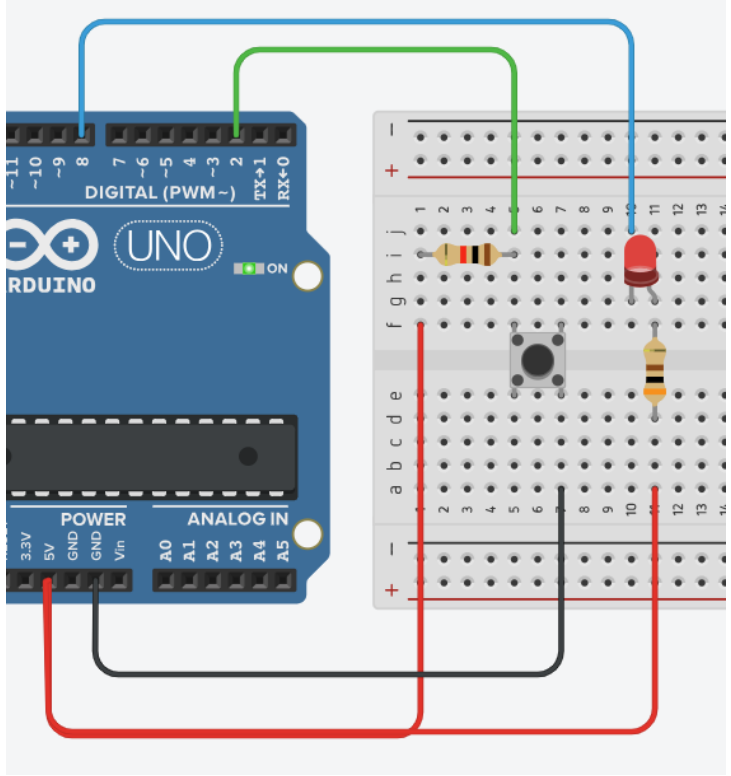
- **INT1 : Interrupt #1**
- **INT0 : Interrupt #0**

외부 인터럽트(External Interrupt)

- 폴링 vs **인터럽트**

2 → INT0 : Interrupt #0

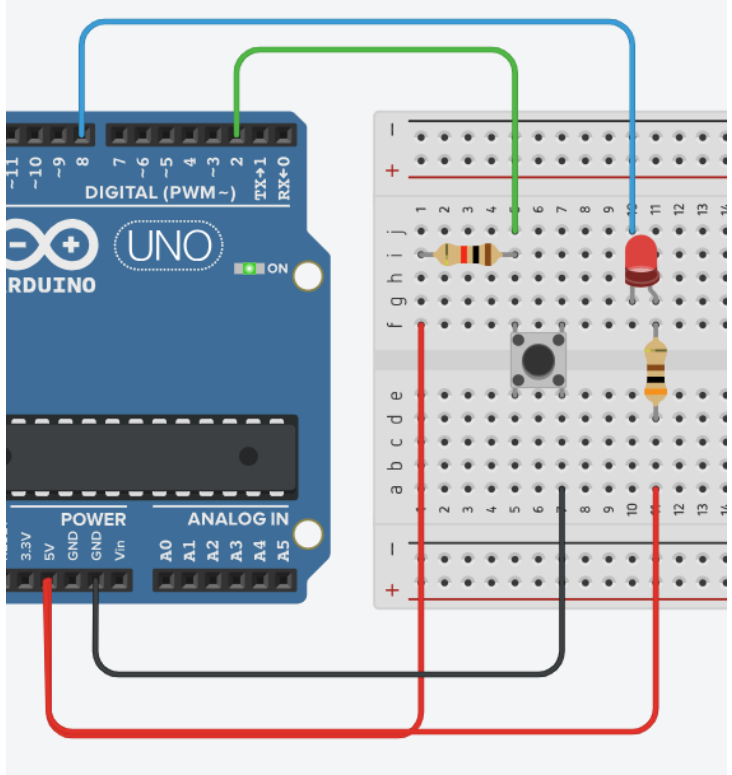




외부 인터럽트(External Interrupt)

- 폴링 vs **인터럽트**

2 → INT0 : Interrupt #0



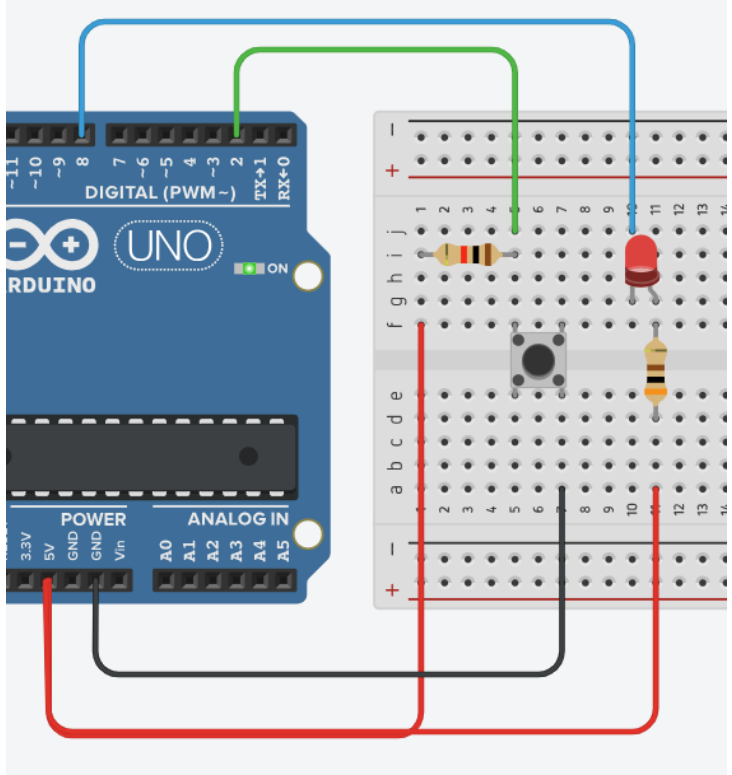
인터럽트 발동 조건 (mode)

모드	상태
LOW	**핀이 LOW일때**
CHANGE	**LOW->HIGH or HIGH->LOW로 변할 때**
RISING	**LOW ->HIGH일때**
FALLING	**HIGH -> LOW일때**
HIGH	**핀이 HIGH일때**

외부 인터럽트(External Interrupt)

- 폴링 vs **인터럽트**

2 → INT0 : Interrupt #0




```
attachInterrupt( digitalPinToInterrupt(핀번호), 서비스루틴함수명, 모드 );
```

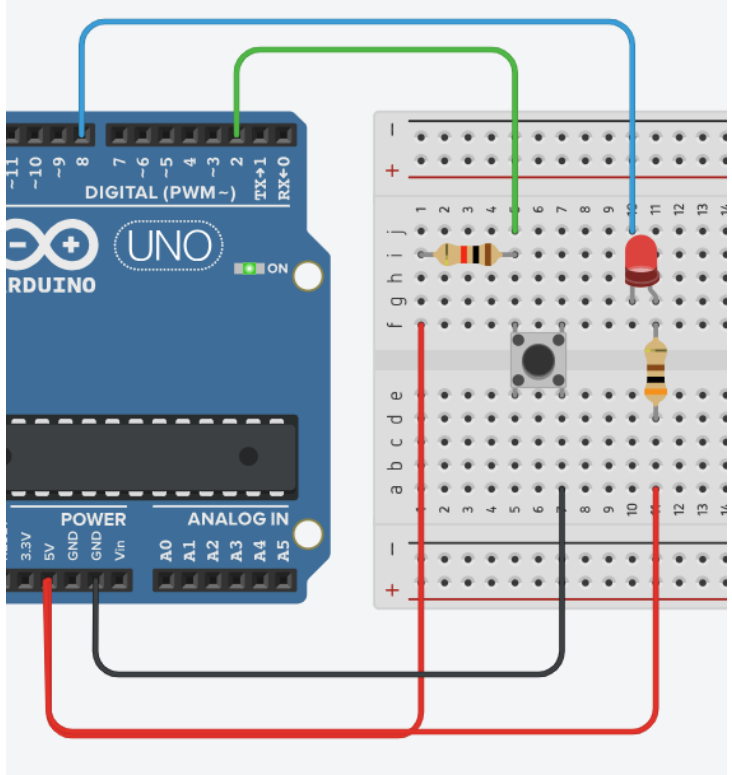
인터럽트 발동 조건 (mode)

모드	상태
LOW	**핀이 LOW일때**
CHANGE	**LOW->HIGH or HIGH->LOW로 변할 때**
RISING	**LOW ->HIGH일때**
FALLING	**HIGH -> LOW일때**
HIGH	**핀이 HIGH일때**

외부 인터럽트(External Interrupt)

- 폴링 vs **인터럽트**

2 → INT0 : Interrupt #0



attachInterrupt(digitalPinToInterrupt(2), ExINT, FALLING);

attachInterrupt(digitalPinToInterrupt(핀번호), 서비스루틴함수명, 모드);

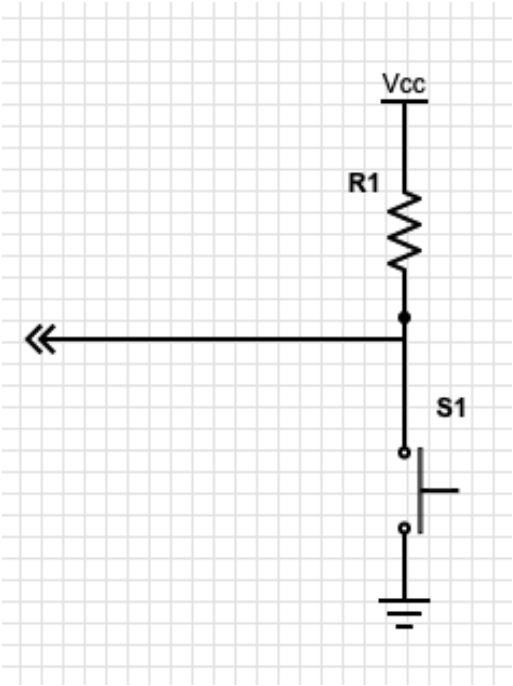
인터럽트 발동 조건 (mode)

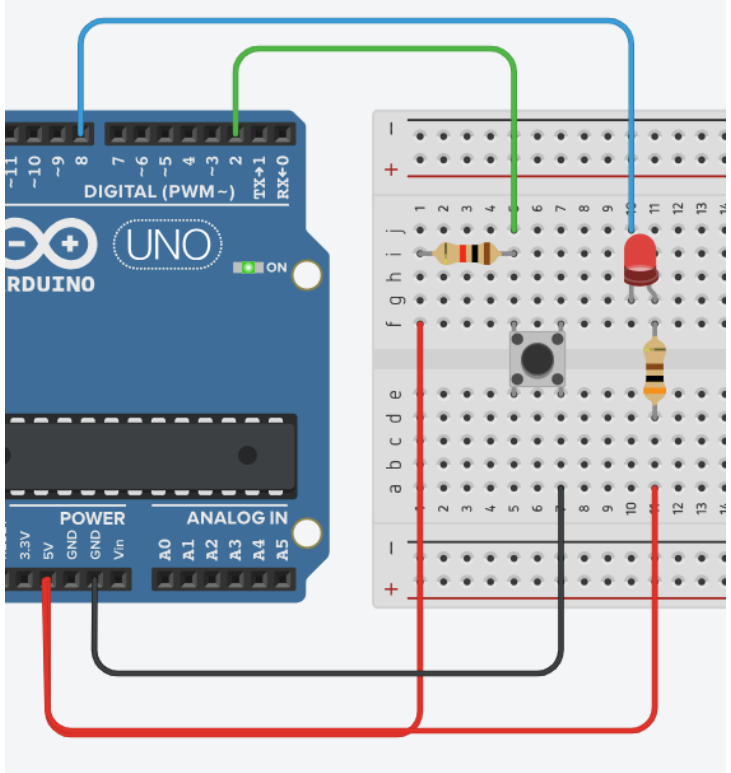
모드	상태
LOW	**핀이 LOW일때**
CHANGE	**LOW->HIGH or HIGH->LOW로 변할 때**
RISING	**LOW ->HIGH일때**
FALLING	**HIGH -> LOW일때**
HIGH	**핀이 HIGH일때**

외부 인터럽트(External Interrupt)

- 폴링 vs **인터럽트**

attachInterrupt(digitalPinToInterrupt(2), ExINT, FALLING);






```
void setup()
{
  pinMode(8, INPUT) ;
  pinMode(2, OUTPUT) ;

  attachInterrupt( digitalPinToInterrupt(2), ExINT, FALLING );

  Serial.begin(9600) ;
}

void loop()
{
  digitalWrite(8, 0) ;
  delay(1000) ;

  digitalWrite(8, 1) ;
  delay(1000) ;
}

void ExINT()
{
  Serial.println("ExINT") ;
}
```